

Building an L2 Finality Benchmark, One Task at a Time

Thirty-four small problems, each with the exact steps to solve it, what to watch out for, and how to know you're done.

Project ID 50081 — Mitacs Globalink 2026

Supervisor Prof. Sara Rouhani (TCDT Lab)

Intern Ujala Kiran

Approach Observational — measure public networks, host nothing

Date 7 August 2026

HOW TO USE THIS DOCUMENT

Every task is written to stand alone. Nothing here assumes a particular existing codebase — the steps work whether you extend code you already have or start from an empty directory.

Work top to bottom. Each task states the problem, the steps, a code sketch where one helps, the pitfalls, and a **done-when** test. Do not move on until the done-when passes: these tasks stack, and a half-finished one three phases back is very expensive to find.

Code is a sketch, not a paste-in. Library APIs shift between versions and rollup RPC methods change. Every snippet shows the shape of the solution; check names against current documentation before relying on them.

§ What you are building

A program that submits a transaction to a Layer 2 network somebody else operates, then determines **three different times** at which that transaction could reasonably be called final — and what it cost to get there.

Level	Final when	Where the timestamp comes from
Full trust t_1	The L2 sequencer accepts it	The L2's own RPC — a transaction receipt appears
Partial trust t_2	Its batch is posted to Ethereum	An Ethereum L1 transaction sent by the rollup operator
Trustless t_3	Its batch is proven or its challenge window closes	A second Ethereum L1 transaction, later

Two of the three timestamps come from Ethereum, not from the rollup. That is the central insight of the whole design and the reason you do not need to host anything.

Dependency order

```
A3 funds → B2 send → C2 map tx to L1 → C3/C4 read  $t_2$ ,  $t_3$  → D real costs
                                     |
                                     └→ C6 two-pass runner → F3 sweep (start early)
```

H1 comparison table — no dependencies — do it on any blocked day

THE THREE TASKS THAT DECIDE WHETHER THIS WORKS

A3 — getting testnet funds. Faucets refuse, rate-limit, and demand accounts you may not have. Start it on day one, in parallel, across every network you might use.

C2 — mapping your L2 transaction to its L1 batch. The genuine technical crux. There is a shortcut; it is described in full.

C6 — separating submission from resolution. Trustless finality can take an hour. If your program waits synchronously, you will lose entire days to a closed laptop.

PHASE A

Unblock

Week 1, days 1-2 · about 6 hours of work, plus however long the faucets take

A1 Set up a clean repository

1 hr

You need a structure that separates the parts that change per rollup from the parts that don't.

STEPS

1. Create the directory layout. Keep it flat; this is a research tool, not a product.
2. Create a virtual environment and install `web3`, `eth-account`, `pyyaml`, `pandas`, `matplotlib`.
3. Freeze exact versions into `requirements.txt` with `==`, not `>=`. A reviewer must be able to recreate your environment.
4. Add a `.gitignore` for `__pycache__`, `.env`, and `results/`.
5. Initialise git and commit.

```
bench/  
  adapters/      # one file per rollup – the only per-network code  
  core/          # submit, resolve, metrics – network-independent  
  configs/       # one YAML per experiment  
  abis/          # committed contract ABIs  
  results/       # gitignored output  
  analysis/      # notebooks and plotting
```

PITFALL

Never commit a private key. Put it in `.env`, load with `os.environ`, and confirm `.env` is ignored *before* your first commit — a key in git history is still there after you delete the file.

DONE WHEN

A fresh clone plus `pip install -r requirements.txt` imports `web3` without error.

A2 Create and record test wallets

30 min

You need an address to fund, and a reproducible way to recreate it.

STEPS

1. Generate one account. Use the **same** account across all networks — addresses are compatible and it keeps your records simple.
2. Write the private key to `.env`. Write the *address* into your config file, committed, so results can be traced.
3. Never put real funds in this account.

```
from eth_account import Account
acct = Account.create()
print(acct.address)           # goes in the config
print(acct.key.hex())         # goes in .env, never in git
```

DONE WHEN

Your program loads the key from the environment and prints the matching address.

A3 Obtain testnet funds

1 hr - 2 days

Every faucet has different rules and some will simply refuse you. This blocks the entire project, so it starts first and runs in parallel with everything else in Phase A.

STEPS

1. Request from **every** faucet you might need on day one — zkSync Sepolia, Polygon zkEVM Cardona, OP Sepolia, Arbitrum Sepolia. Do not wait to find out which network you prefer.
2. If a faucet demands a mainnet balance or an aged account, take the alternate route: get Sepolia ETH from an Ethereum faucet, then **bridge** it to the L2 through the rollup's official bridge UI. Slower, but it does not gatekeep.
3. Request more than you think you need. A thousand transactions at testnet gas is still small, and re-requesting costs you a day.
4. Record which faucet, when, and how much — this goes in your methodology.

PITFALL

Bridging L1 to L2 is itself slow, sometimes fifteen minutes or more. Start it and move on to A4; do not sit watching it.

DONE WHEN

A non-zero balance is visible for your address on each L2's block explorer.

A4 Verify every RPC endpoint

1 hr

Public endpoints throttle, and you need to know their limits before they surprise you mid-experiment.

STEPS

1. For each network, connect and fetch the current block number and chain ID.
2. Confirm the chain ID matches the network you think you are on. This catches configuration mistakes that would otherwise look like inexplicable failures much later.
3. Register a free-tier Alchemy or Infura key now. You will need it in Phase C, where queries over wide block ranges trigger rate limits fastest.
4. Put every endpoint in a config file, never inline in code.

```
from web3 import Web3
w3 = Web3(Web3.HTTPProvider(url))
assert w3.is_connected()
print(w3.eth.chain_id, w3.eth.block_number)
```

DONE WHEN

A single script prints block number and chain ID for every network in your config.

A5 Send one transaction by hand

2 hr

Before any framework exists, prove that keys, funds, endpoint and chain ID all work together. When something breaks later you want to know it is not this.

STEPS

1. Write a throwaway script — twenty lines, no abstraction — that sends a tiny transfer from your account to itself.
2. Print the hash and open it in the block explorer.
3. Keep the script. It becomes your reference when the framework version misbehaves.

```
tx = {
    "to": acct.address,
    "value": w3.to_wei(0.0001, "ether"),
    "nonce": w3.eth.get_transaction_count(acct.address),
    "gas": 100_000,
    "gasPrice": w3.eth.gas_price,
    "chainId": w3.eth.chain_id,
}
signed = acct.sign_transaction(tx)
h = w3.eth.send_raw_transaction(signed.raw_transaction)
print(h.hex())
```

PITFALL

The signed object's attribute is `raw_transaction` in web3.py v7 and `rawTransaction` in v6. If you get an `AttributeError`, that is why.

DONE WHEN

The transaction appears in the explorer with status Success.

Make it real

Week 2 · the keystone — everything downstream depends on this phase

B1 Define the workload contracts

1 day

You need transactions that represent real usage, not just empty transfers — and their calldata must be correct or every run fails at once.

STEPS

1. Start with **two** workloads only: a native transfer and an ERC-20 transfer. Add more later if time allows.
2. For the ERC-20, deploy a standard OpenZeppelin token to the testnet once, and record its address in your config.
3. Compute function selectors with a tool, never by hand. `cast sig "transfer(address,uint256)"` gives you the four bytes.
4. Encode arguments with web3.py's contract interface rather than string concatenation.

```
contract = w3.eth.contract(address=token, abi=erc20_abi)
tx = contract.functions.transfer(to, 1).build_transaction({...})
```

PITFALL

Hand-written calldata is where silent corruption lives — a selector containing a non-hex character will pass every check you write and fail on the first real submission. Let the ABI encoder do it.

DONE WHEN

Both workloads produce transaction dictionaries that a dry-run gas estimate accepts.

B2 Implement real submission

1–2 days

This is the keystone. Until a real hash comes back from a real node, nothing else in this document can be built.

STEPS

1. Write a `submit(tx)` function: sign, broadcast, return the hash the node gives you.
2. Record the wall-clock time **immediately before** broadcast. This is `t0` and every latency in your study is measured from it.
3. Return a record — hash, workload name, `t0` — not just a hash. Everything downstream needs the context.
4. Estimate gas per transaction rather than hardcoding a limit; different workloads need very different amounts.

```
import time

def submit(w3, acct, tx):
    tx["gas"] = w3.eth.estimate_gas(tx)
    signed = acct.sign_transaction(tx)
    t0 = time.time()
    h = w3.eth.send_raw_transaction(signed.raw_transaction)
    return {"hash": h.hex(), "t0": t0, "workload": tx["_name"]}
```

NEVER FABRICATE A HASH

If submission fails, raise or record the failure. Returning a placeholder that later code polls for is the single most damaging thing you can do to a benchmark: every downstream measurement becomes plausible-looking and meaningless, and nothing warns you.

DONE WHEN

A hash produced by your framework resolves in the block explorer.

B3 Manage nonces under concurrency

0.5 day

Transactions from one account must carry consecutive nonces. Fetching the nonce inside each concurrent task gives several transactions the same number, and all but one are rejected.

STEPS

1. Fetch the account's transaction count **once**, before the batch begins.
2. Assign `base + i` locally to each transaction in the batch.
3. Submit in index order. A gap stalls everything behind it, so ordering matters even when sending concurrently.
4. After each batch, re-fetch the count and confirm it advanced as expected. If not, a transaction was dropped — record it and re-sync before the next batch.

```
base = w3.eth.get_transaction_count(acct.address)
for i, tx in enumerate(batch):
    tx["nonce"] = base + i
```

PITFALL

If this fights you, submit strictly sequentially at first. Slower, entirely correct, and concurrency can be reintroduced in Phase F once everything else works. Do not let a nonce bug consume the keystone week.

DONE WHEN

Fifty transactions submitted in one batch all appear on-chain, none rejected for nonce reasons.

B4 Measure full-trust finality, t_1

1 day

The first of your three timestamps: the moment the L2 acknowledges the transaction.

STEPS

1. Poll `eth_getTransactionReceipt` against the L2 until it returns something other than `None`.
2. Use exponential backoff — 1s, 2s, 4s, capped around 10s — not a tight loop. Public endpoints will throttle you otherwise.
3. Set a deadline, perhaps 120 seconds, and record a timeout as a **failure with a reason**, never as a large latency.
4. Record the wall-clock time when the receipt first appeared. That is t_1 .

```
def wait_receipt(w3, h, deadline=120):
    delay, start = 1.0, time.time()
    while time.time() - start < deadline:
        r = w3.eth.get_transaction_receipt(h)  # None until mined
        if r is not None:
            return time.time(), r
        time.sleep(delay)
        delay = min(delay * 2, 10)
    raise TimeoutError(h)
```

PITFALL

Some web3.py versions raise `TransactionNotFound` rather than returning `None`. Catch both.

DONE WHEN

$t_1 - t_0$ **varies** between runs. A constant value means you are measuring your own polling interval, not the network.

B5 Handle failures honestly

0.5 day

Transactions revert, run out of gas, and get dropped. Silently discarding them biases every statistic you publish.

STEPS

1. Check `receipt["status"]`. A value of 0 is a revert — on-chain, but failed.
2. Give every record an explicit outcome: `success`, `reverted`, `timeout`, or `rejected`.
3. Compute latency statistics over successes only, and **report the success rate alongside them**.
4. Group identical failures when printing. A hundred copies of one message hides the one that differs.

DONE WHEN

A deliberately under-gassed transaction is reported as failed with a reason, and is excluded from the latency figures.

The contribution

Weeks 3–4 · your actual research, and the hardest part of the build

C1 Collect contract addresses and ABIs

2 hr

Two of your three timestamps come from an Ethereum contract, so you need to be able to read it.

STEPS

1. Find the rollup's L1 contract address for your **testnet** — this differs from mainnet. The rollup's own documentation is the authoritative source; the L1 explorer confirms it.
2. Download the ABI from Etherscan and **commit it** to `abis/`. Fetching at runtime makes your results depend on a third party being up.
3. Record the address in your config, not in code.

DONE WHEN

You can construct a contract object from the committed ABI and call a read-only method on it.

C2 Map your transaction to its L1 settlement

1 day

You have an L2 transaction hash. You need to know which Ethereum transaction posted its batch, and which one proved it. This is the crux of the entire project.

TAKE THE SHORTCUT FIRST

Both major ZK rollups expose custom RPC methods that hand you this mapping directly, so you can avoid scanning event logs entirely. Try this before writing any log-parsing code — it can collapse C2, C3 and C4 into a few polls.

STEPS — PREFERRED ROUTE

1. Call the rollup's transaction-details method with your L2 hash. On zkSync Era this is `zks_getTransactionDetails`, which returns fields for the L1 commit, prove and execute transaction hashes. On Polygon zkEVM, find the batch with `zkevm_batchNumberByBlockNumber`, then fetch it with `zkevm_getBatchByNumber`, which carries the sequence and verify transaction hashes.
2. These fields are `null` until each stage completes. Poll until populated.
3. Once you have an L1 hash, fetch that L1 transaction, read its `blockNumber`, fetch the block, and take its `timestamp`. **That timestamp is your finality time.**
4. Store the L1 hashes in your output. They are what make your results verifiable by anyone.

```
res = w3_l2.provider.make_request("zks_getTransactionDetails", [tx_hash])
d = res["result"]
# d["ethCommitTxHash"] -> t2 source
# d["ethProveTxHash"] -> t3 source

l1_tx = w3_l1.eth.get_transaction(d["ethCommitTxHash"])
t2 = w3_l1.eth.get_block(l1_tx["blockNumber"])["timestamp"]
```

STEPS — FALLBACK ROUTE

1. If the RPC method is unavailable, watch the L1 contract's event logs instead: the batch-commit event and the proof-verification event, each carrying a batch number.
2. Query logs in **bounded block ranges** — a few thousand blocks at a time. Unbounded queries are the fastest way to get rate-limited.
3. Match your transaction's batch number against the numbers in the events.

```
logs = contract.events.BlockCommit().get_logs(  
    from_block=start, to_block=start + 2000  
)
```

PITFALL

RPC method names and returned field names change between rollup versions. Verify against current documentation rather than trusting any snippet, including this one.

Timebox this to two days. If the preferred route resists, switch to the fallback. If both resist, record the batch number and estimate finality from observed batch cadence — declaring the estimate in your methodology. A declared estimate is a limitation; an undeclared one is a fabrication.

DONE WHEN

For one L2 transaction you can print two Ethereum transaction hashes, and both open in the L1 explorer.

C3 Measure partial-trust finality, t_2

1 day

The moment the batch containing your transaction was posted to Ethereum.

STEPS

1. Using C2, obtain the commit transaction hash and its L1 block timestamp.
2. Record t_2 as that block timestamp.
3. Store the L1 hash and block number alongside it.

PITFALL

Block timestamps have second granularity and are set by the proposer, so they are not perfectly precise. Say so in your threats-to-validity section — it is a minor effect against intervals measured in minutes, but a reviewer will notice if you do not mention it.

DONE WHEN

$t_2 - t_0$ lands in the range of minutes and differs across transactions submitted at different points in a batch interval.

C4 Measure trustless finality, t_3

1 day work · hours of waiting

The moment the batch was proven on Ethereum — or, for an optimistic rollup, the moment its challenge window closed.

STEPS

1. For a ZK rollup, take the proof-verification transaction from C2 and use its L1 block timestamp.
2. Do not hold a connection open waiting for this. It is what C6 exists to solve.
3. Record whether the batch was proven or merely executed if the rollup distinguishes these — and state which one you used as t_3 .

TESTNET FINALITY IS NOT MAINNET FINALITY

Testnets prove more frequently and, for optimistic rollups, use drastically shortened challenge windows — around an hour where mainnet takes seven days. Record the network and the observed interval with every figure, and pair your testnet measurements with read-only mainnet observation (task F5) so you can state the real-world number honestly.

DONE WHEN

You have a t_3 derived from an L1 transaction hash that anyone can independently check.

C5 Measure each interval independently

2 hr

If each level is timed from the end of the previous one, or measured cumulatively without care, t_3 silently contains t_2 and t_1 and every trustless figure is inflated.

STEPS

1. Store four **absolute** timestamps per transaction: t_0 , t_1 , t_2 , t_3 .
2. Compute all three latencies from t_0 at analysis time, never during collection.
3. Keep the raw timestamps in your output so any error here is correctable afterwards rather than fatal.

```
{"hash": "0x...", "t0": 1754.., "t1": 1754.., "t2": 1754.., "t3": 1754..}  
# latencies derived later: t1-t0, t2-t0, t3-t0
```

DONE WHEN

Your output file contains absolute timestamps and no pre-computed durations.

C6 Split submission from resolution

0.5 day

Trustless finality can take an hour or more. A program that waits synchronously ties up your machine, dies when the laptop sleeps, and loses everything it had collected.

STEPS

1. Write **two** programs instead of one.
2. **Submit pass:** sends transactions, writes one JSON object per line to a file with hash, workload, t_0 and t_1 , then exits. Runs in minutes.
3. **Resolve pass:** reads the file, finds rows still missing t_2 or t_3 , queries for them, writes the file back. Safe to run repeatedly.

4. Use JSON Lines — one object per line. It survives interruption, appends cheaply, and a partly-written file is still mostly readable.
5. Run the resolve pass on a schedule, or simply by hand each morning.

```
# submit pass — fast, then exits
with open("run_042.jsonl", "a") as f:
    for rec in submitted:
        f.write(json.dumps(rec) + "\n")

# resolve pass — run later, repeatedly, until complete
for rec in rows:
    if rec.get("t3") is None:
        rec["t3"] = try_resolve_t3(rec["hash"])
```

WHY THIS MATTERS MORE THAN IT LOOKS

This one design decision lets you begin collecting data in Week 3 and keep collecting it while you build Phase D, E and F. Without it, every experiment costs its full wall-clock duration in blocked time — and you do not have those hours to spend.

DONE WHEN

You can kill the process mid-run, restart, and lose nothing.

Make it defensible

Week 4 · cheap work, and the minimum viable deliverable sits at the end of it

D1 Export raw per-transaction data

3 hr

Without transaction hashes in your output, nobody — including you in six weeks — can verify a single number you report.

STEPS

1. Write one row per transaction containing: L2 hash, both L1 hashes, all four timestamps, workload name, gas used, outcome, network, and run identifier.
2. Emit summary statistics as a *separate* file derived from the raw rows, never as a replacement for them.
3. Copy any result you intend to cite in the report into a permanent directory. Treat `results/` as scratch.

DONE WHEN

You can pick any figure in your report, find the rows behind it, and open one of them in a block explorer.

D2 Compute real L1 costs

0.5 day

Cost per transaction must come from the actual Ethereum transactions that settled your batch, not from constants.

STEPS

1. From each L1 receipt take `gasUsed` and `effectiveGasPrice`. Their product is the cost in wei.
2. Convert to ETH, then to USD using a rate you fetch and **record with a timestamp**.
3. Divide the batch's cost by the number of transactions in that batch to get a per-transaction share.
4. Print the gas price and ETH rate beside every dollar figure you output.

```
cost_wei = r["gasUsed"] * r["effectiveGasPrice"]
cost_usd = w3.from_wei(cost_wei, "ether") * eth_usd
```

A DOLLAR FIGURE WITHOUT ITS ASSUMPTIONS IS UNFALSIFIABLE

Two people running the same experiment a week apart will get different costs purely from gas and ETH price movement. Recording the assumptions is what makes the comparison meaningful — and what stops a reviewer dismissing the number entirely.

DONE WHEN

Changing the ETH price in your config changes the reported cost, and the report states the price it used.

D3 Measure data-availability size

1 day

How many bytes the rollup posted to Ethereum for your batch — the dominant cost term for most rollups, and the thing EIP-4844 changed.

STEPS

1. Fetch the L1 batch-posting transaction and check its type. Type 3 carries blobs; earlier types carry calldata.
2. For blobs: count the blob hashes and multiply by 131,072 bytes each. Price using the blob gas price.
3. For calldata: take the length of the input data and price at 16 gas per non-zero byte.
4. Report bytes and cost separately — the byte count is a fact about the rollup, the cost depends on your price assumptions.

PITFALL

Never return a fixed byte count regardless of workload. If you cannot get the real size, report the field as unavailable rather than filling it with a plausible number.

DONE WHEN

DA bytes differ between a native transfer batch and an ERC-20 batch.

D4 Compute throughput honestly

3 hr

Throughput means different things at each finality level, and the number you can legitimately report is narrower than the one people usually quote.

STEPS

1. Report **achieved** throughput: successful transactions divided by elapsed wall time, at your chosen submission rate.
2. Do not describe it as peak or maximum throughput. You did not saturate the network and must not attempt to.
3. Where the project requires peak figures, cite published values with their source and say plainly that they are cited.

DO NOT STRESS-TEST A SHARED PUBLIC NETWORK

Finding a sequencer's breaking point means degrading a service other people depend on. Beyond the ethics, you would be measuring your own rate limits rather than the system. This limitation is worth one honest paragraph in your report.

DONE WHEN

Every throughput figure in your output carries a finality level and the word "achieved".

D5 Remove anything that silently does nothing

2 hr

Configuration options that are parsed and then ignored are the worst class of bug in a benchmark: you change a setting, the run completes, the number is unchanged, and nothing tells you.

STEPS

1. For each config key, trace whether anything actually reads it.
2. Delete the dead ones, or raise an explicit "not implemented" error.
3. Do the same for any documentation claim you cannot demonstrate on request.

DONE WHEN

Every key in your config file changes something observable, or refuses to run.

MINIMUM VIABLE DELIVERABLE — YOU ARE HERE

With Phases A through D complete you have: real transactions with verifiable hashes; latency at three finality levels; cost from real L1 gas with stated assumptions; and raw exported data.

That is a complete, defensible piece of research. **If everything after this collapses, write it up and deliver it.**

The comparison

Week 5 · the most interesting result, and the first thing to cut

E1 Add an optimistic rollup

2–3 days

Everything so far measures ZK rollups. The comparison only becomes interesting with a fundamentally different architecture beside it.

STEPS

1. Choose OP Sepolia or Arbitrum Sepolia. Submission and t_1 work exactly as before — the L2 RPC is Ethereum-compatible.
2. For t_2 , find the batcher transaction on L1 that posted your batch. Optimistic rollups do not expose this as conveniently as zkSync does; expect to identify the batcher address and match by block range.
3. For t_3 , the semantics change entirely — see E2.
4. Keep the adapter interface identical so the rest of your code does not care which rollup it is talking to.

DONE WHEN

The same workload runs unmodified against both a ZK and an optimistic rollup.

E2 Model challenge-window finality

1 day

For a ZK rollup, trustless finality is a proof being verified. For an optimistic rollup it is the absence of a successful challenge before a deadline. These are not the same measurement and must not be presented as though they were.

STEPS

1. Identify when the output root covering your transaction was proposed on L1.
2. Add the network's challenge period to that time. That sum is t_3 .
3. Mark this value in your data as **derived**, not observed — you are computing a deadline, not witnessing an event.
4. Record both the testnet challenge period and the mainnet value, and report both.

THIS ASYMMETRY IS THE FINDING

An optimistic rollup looks excellent at full trust and dramatically worse at trustless finality, while a ZK rollup changes far less between the two. That inversion is what none of the papers on your reading list produced, and it is the single most valuable result available to you. Protect the work that produces it.

DONE WHEN

Your data distinguishes observed timestamps from derived ones, and the report explains why.

STEPS

1. Same workload, same batch size, submitted to both rollups as close together in time as you can manage.
2. Record L1 gas price at the time of each run — a comparison across very different gas conditions is not a comparison.
3. Produce a single table: rollup × finality level × latency and cost.

DONE WHEN

One table shows both rollups at all three finality levels.

Data collection

Week 6 · little work, much waiting — start the long runs as early as C6 allows

F1 Design the experiment matrix

0.5 day

STEPS

1. Write down every combination before running anything: rollup × workload × batch size × repetition.
2. Keep it small. Two rollups × two workloads × three batch sizes × five repetitions is 60 runs and is plenty.
3. Give each cell an identifier and record it with every row of output.
4. Commit one YAML config per cell so any run can be repeated from the file.

DONE WHEN

A checklist exists with one line per run, and you can tick them off.

F2 Repeat every configuration

0.5 day

A single run produces a single number, and percentiles computed over one value are meaningless.

STEPS

1. Five repetitions minimum per configuration.
2. **Separate them in time** — an hour apart at least. Five runs back to back measure one moment's network conditions five times.
3. Report the median and the spread, not the mean alone.
4. Keep every repetition's raw data.

DONE WHEN

Every reported figure has at least five underlying runs, spread across time.

F3 Sweep batch size

1 day work · several days waiting

Batch size drives the trade-off between fast finality and amortised cost — the most interesting single variable you control.

STEPS

1. Choose three or four sizes, well separated: 1, 10, 100, perhaps 500.
2. Submit each, then let the resolve pass fill in finality over the following days.
3. **Start this the moment C6 works**, even with Phases D and E unfinished. The data ages perfectly well while you build.
4. Plot per-transaction cost against batch size, and finality latency against batch size, on the same axis.

DONE WHEN

Per-transaction cost falls with batch size and you can point at where it stops falling usefully.

F4 Observe mainnet, read-only

1 day

Your testnet numbers are real but not representative. Mainnet observation costs nothing and grounds them.

STEPS

1. Point the same event-reading code at mainnet rollup contracts. Send no transactions and spend nothing.
2. Collect batch commit and proof events over a week or two.
3. Derive real batch intervals, proof cadence, DA sizes and costs under production load.
4. Present these beside your testnet figures to show how far the testnet diverges.

A STRONG SUBSTITUTE IF TIME RUNS SHORT

If Phase E never happens, mainnet observation still lets you compare optimistic and ZK rollups on batch cadence, proof interval and cost — without submitting a single transaction. It is the cheapest remaining source of a real comparison.

DONE WHEN

You can state observed mainnet batch and proof intervals with the block range you measured them over.

Analysis

Week 7 · turning rows into findings

G1 Plot latency distributions

1 day

STEPS

1. Load the raw JSONL into pandas. Compute the three latencies from absolute timestamps here, not during collection.
2. Draw a cumulative distribution per rollup per finality level. CDFs show the tail; bar charts of means hide it.
3. Use a log scale on the time axis — your three levels span seconds to hours and will not share a linear axis.
4. Report median, 95th and 99th percentiles in a table beside the plot.

DONE WHEN

One figure shows all three finality levels for one rollup, with the tail visible.

G2 Build the finality inversion figure

0.5 day

This is your headline result. Everything else in the report supports it.

STEPS

1. Take the **same** transaction set. Plot latency for both rollups, grouped by finality level.
2. Order the groups full trust → partial trust → trustless so the crossover reads left to right.
3. Annotate the crossing point directly on the figure.
4. Write the caption as a claim, not a description: state which rollup wins at which definition and by how much.

SKETCH THIS IN WEEK 3

Draw it on paper before you have the data. Knowing exactly which figure you are building tells you precisely which columns your collection code must produce — and stops you discovering a missing field in Week 7.

DONE WHEN

A reader who sees only this figure understands the paper's argument.

G3 Break down the cost

0.5 day

STEPS

1. Split per-transaction cost into DA, batch posting and proof verification.
2. Stacked bars per rollup, with the gas price and ETH rate in the caption.
3. State which component dominates, and at which batch size that changes.

DONE WHEN

The components sum to the total, and the assumptions appear on the figure.

G4 Check reproducibility

1 day

STEPS

1. Re-run two or three configurations on a different day, from the committed config files.
2. Compare against the original results and quantify the difference.
3. If they diverge substantially, that is a finding about network variability — report it rather than hiding it.

DONE WHEN

You can state how much your results vary between days, with a number.

G5 Write threats to validity

0.5 day

The section that makes a reviewer trust the rest. Write it honestly and early.

MUST INCLUDE

- Testnet challenge periods and proof cadence differ from mainnet.
- Peak throughput was not measured; achieved throughput is reported instead.
- Proving time and cost are cited from published work, not measured.
- Public RPC endpoints introduce variable latency outside your control.
- Submission came from one machine in one geographic region.
- L1 block timestamps have second granularity and are proposer-set.
- Optimistic trustless finality is derived from a deadline, not observed.

DONE WHEN

Every limitation you know of is written down before anyone else finds it.

Write and deliver

Week 8 · except H1, which starts today

H1 Build the framework comparison table

1 day · zero dependencies

Your related-work section, and the clearest statement of your research gap. It needs no code, no data and no funds — so it is what you do whenever something else is blocked.

STEPS

1. Rows: Blockbench, Hyperledger Caliper, BCTMark, Gromit, the Cable thesis, Chaliasos et al. (AFT 2024), and yours.
2. Columns: targets L1 or L2, metrics covered, finality-aware, network emulation, measures cost, reproducible, code public.
3. Fill every cell from the papers themselves, not from memory.
4. **The empty cells in the "finality-aware" column are your contribution.** Say so explicitly in the surrounding text.

EVERY CITATION GETS A DOI YOU HAVE PERSONALLY OPENED

Fabricated references are the fastest way to lose a reviewer's trust in everything else you wrote. Check each one. Keep them in Zotero, and cite nothing you have not held.

DONE WHEN

The table is complete and every row cites a real, verified publication.

H2 Write the report

3 days · drafted throughout

STRUCTURE

1. **Introduction** — why L2 finality measurement matters. Half a page.
2. **Background** — rollups, finality levels, EIP-4844. Two pages.
3. **Related work** — the H1 table plus discussion. Two pages.
4. **Methodology** — the observational design, and why hosting was infeasible. Draft this in Week 3 while the reasoning is fresh. Three pages.
5. **Implementation** — architecture, adapters, the two-pass runner. Two pages.
6. **Results** — the G-phase figures with discussion. Four pages.
7. **Threats to validity** — G5. One page.
8. **Conclusion and future work** — what a GPU and a testbed would unlock. One page.

PITFALL

Do not leave writing to Week 8. Draft methodology in Week 3 and related work whenever you are blocked. Week 8 is for assembling and polishing, not composing from nothing.

DONE WHEN

Someone outside the project can read it start to finish and explain what you measured and why.

H3 Write a README a stranger can follow

0.5 day

STEPS

1. Install steps, configuration, one worked example run, and where output lands.
2. State plainly what the framework does **not** measure.
3. Then test it: follow your own instructions in a fresh clone and fix whatever breaks.

DONE WHEN

A clean checkout, following only the README, produces a result.

H4 Prepare the presentation

0.5 day

STEPS

1. Ten to twelve slides. One question, one method, three findings, honest limitations.
2. Lead with the G2 inversion figure. It is the thing people will remember.
3. Rehearse the limitations slide most carefully — that is where the questions come from.

DONE WHEN

You can deliver it in the allotted time without reading the slides.

H5 Write handover notes

0.5 day

STEPS

1. What works, what does not, and what you would do next with more time or a GPU.
2. Every dead end you hit and why — this is the most valuable thing you can leave behind.
3. Where the data lives and how to regenerate it.

DONE WHEN

The next person could resume your work without speaking to you.

§ Quick reference

ID	Task	Effort	Done when
A1	Clean repository	1 hr	Fresh clone installs and imports
A2	Test wallets	30 min	Key loads from environment
A3	Testnet funds	1 hr – 2 days	Balance visible on explorer
A4	Verify endpoints	1 hr	Block number and chain ID for each
A5	Manual transaction	2 hr	Success status in explorer
B1	Workload contracts	1 day	Gas estimate accepts both workloads
B2	Real submission	1–2 days	Framework hash resolves in explorer
B3	Nonce management	0.5 day	50 concurrent, none rejected
B4	Full-trust finality t_1	1 day	Latency varies between runs
B5	Failure handling	0.5 day	Reverts excluded, reasons recorded
C1	Addresses and ABIs	2 hr	Read-only call succeeds
C2	Map tx to L1 settlement	1 day	Two L1 hashes open in explorer
C3	Partial-trust finality t_2	1 day	Minutes, varies within batch interval
C4	Trustless finality t_3	1 day	Derived from a checkable L1 hash
C5	Independent intervals	2 hr	Output stores absolute timestamps
C6	Two-pass runner	0.5 day	Kill mid-run, lose nothing
D1	Raw export	3 hr	Any figure traceable to a hash
D2	Real L1 costs	0.5 day	Config price changes the result
D3	DA size	1 day	Bytes differ by workload
D4	Honest throughput	3 hr	Every figure labelled "achieved"
D5	Remove dead config	2 hr	Every key does something
E1	Optimistic adapter	2–3 days	Same workload, both architectures
E2	Challenge-window model	1 day	Derived values marked as derived
E3	Cross-architecture run	1 day	One table, both rollups, three levels
F1	Experiment matrix	0.5 day	Tickable checklist exists
F2	Repetitions	0.5 day	5+ runs, spread across time
F3	Batch sweep	1 day + wait	Cost falls, plateau identified
F4	Mainnet observation	1 day	Intervals stated with block range
G1	Latency CDFs	1 day	Three levels, tail visible
G2	Inversion figure	0.5 day	Figure alone conveys the argument
G3	Cost breakdown	0.5 day	Components sum, assumptions shown

G4	Reproducibility	1 day	Day-to-day variance quantified
G5	Threats to validity	0.5 day	All known limits written down
H1	Comparison table	1 day	Complete, every citation verified
H2	Report	3 days	An outsider can follow it
H3	README	0.5 day	Clean checkout produces a result
H4	Presentation	0.5 day	Delivered without reading slides
H5	Handover	0.5 day	Next person resumes unaided

If the schedule collapses

The smallest set that still yields an honest report: **A1-A5, B2, B4, C2, C3, D1, H1**. That gives real transactions with verifiable hashes, two of three finality levels, raw exported data, and the literature positioning — roughly three focused days once funds are in hand.

Rules that hold throughout

- ☐ Never invent a number. Report a field as unavailable instead.
- ☐ Never publish a cost without the gas price and ETH rate beside it.
- ☐ Never report a latency without saying which finality level it is.
- ☐ Keep raw per-transaction data for everything. Summaries are derived, never primary.
- ☐ Pin versions, commit configs, record endpoints.
- ☐ Cite nothing you have not opened.
- ☐ When blocked, work on H1.